Think about it; most of the time you don't need all of those options. If you partially apply the function (discussed in chapter 6) and create a new function that has fewer arguments, then you can make it more manageable. Instead of having lots of dependencies,

```
visits = (database, http, user, permissions) ->
  if permissions.allowDataFor user
    http.hitsFor database.userIdFor(user.name)
  else
    'private
```

make a `makeVisits` function that fixes the `user` and `permissions` arguments to `visits`:

```
makeVisitsForUser = (database, http) ->
  (user, permissions) ->
    visits database, http, user, permissions
```

This function would typically be used dynamically, but an example of using it for a single known user, by explicitly passing in a user and permissions, is as follows:

```
database = new Database
http = new Http
```

database and http created from a class. It's likely that these wouldn't be created using classes; the classes are used here as examples.

```
visitsForUser = makeVisitsForUser bob, permissions
```

Getting a two-argument function as visitsForBob.

The resulting `visitsForBob` function can then be used:

```
bob = new User
permissionsForBob = new Permissions
```

Examples of how user and permissions might be created.

```
visitsForUser bob, permissionsForBob
```

Invoke the visitsForUser function to get the visits for bob.

Cute, you think, but how is it useful in a test? Imagine you have two tests that use the `visits` function. Each of them creates doubles for `database`, `http`, `user`, and `permissions`:

```
fact 'Visits not shown when permissions are private', ->
  database = databaseDouble
  http = httpDouble
```